

Hosting the Zava Agentic Substrate in Your Azure — A One-Page Brief

Audience: any delivery partner or internal platform team standing up the Zava control plane in a customer's (or their own) Azure tenant for evaluation purposes.

Bottom line: don't write anything. The recipe already exists as a published skill. This brief lists the small set of prerequisites you must provision in the target tenant and points you at the canonical instructions.

The recipe is already written — use it

The MS GBB skills catalogue ships a three-skill pipeline. Overview & visuals: <https://aiappsgbb.github.io/awesome-gbb/zava/>

Order	Skill	Purpose	Wall clock
1 (optional)	research-company	Profile a target company from its public footprint into an org-brief.yaml	15–40 min
2 (optional)	compose-org	Fork the substrate into a customer-branded clone (rebrand, named ELT, vertical entity kinds, vertical workflow stubs)	30–60 min
3 (required)	zava-workspace-deploy	azd up the substrate (branded or generic) to Azure Container Apps	~10 min

Skip steps 1–2 if a generic-looking demo is fine. Run them if you want the workspace to look like the customer's org.

Skills repo: <https://github.com/aiappsgbb/awesome-gbb> **Deploy skill:** skills/zava-workspace-deploy/SKILL.md

Substrate repo: <https://github.com/arturcrambot/zava-control-plane>

What the deploy skill creates (workload-scoped, one resource group)

User-Assigned Managed Identity · Storage Account + Azure Files share (KuzuDB persistence) · Container Apps managed environment · the Container App itself (single uvicorn process serving FastAPI + 3 React SPAs on port 80) · AcrPull role assignment from the UAMI onto the shared ACR.

What the skill does NOT create — you must provision these first

The deploy skill assumes "shared infra" already exists in the subscription. Stand these up once per subscription before running azd up :

#	Component	Purpose	Skill expects
1	Azure subscription + target resource group	Where the workload lands	Owner rights on the deploy account
2	Azure Container Registry (Basic SKU is fine)	Hosts the built image	AZURE_ACR_LOGIN_SERVER , AZURE_ACR_NAME , AZURE_ACR_RESOURCE_GROUP

#	Component	Purpose	Skill expects
3	Application Insights (+ its Log Analytics workspace)	OTEL + container logs + AGT governance events	<code>APPLICATIONINSIGHTS_CONNECTION_STRING</code>
4	LLM endpoint — either an APIM gateway URL or a direct Azure OpenAI / Foundry endpoint	The agents call OpenAI-shape APIs	<code>AZURE_OPENAI_ENDPOINT</code> , deployment names for <code>gpt-4.1</code> and <code>text-embedding-3-large</code>
5	Foundry AI Services account (only if hosting your own models)	Model deployments	Manual RBAC: grant the workload UAMI <code>Cognitive Services OpenAI User + Foundry User</code> on this account
6	Entra app registration (post-deploy)	SSO via Container Apps Easy Auth on the public ingress	Restrict to the target tenant's users

That's the whole list. No firewall, no private endpoints, no WAF, no managed DB, no Sentinel content — the substrate runs against embedded KuzuDB on the Azure Files share and that's good enough for evaluation.

Idle cost envelope: **~€100–500 / month** (the Container App scales down; ACR Basic + App Insights + Storage are the floor).

Tenant isolation (mandatory)

The deploy skill mandates the `azure-tenant-isolation` skill (also in `awesome-gbb`), which sets per-tenant `AZURE_CONFIG_DIR` and `AZD_CONFIG_DIR` so an engineer with access to multiple Azure tenants cannot accidentally deploy into the wrong one. Configure once per workstation before any `az` / `azd` command.

End-to-end steps

1. Install `az` ≥ 2.60 , `azd` (Azure Developer CLI), Docker, Node 20, Python 3.13, `uv`.
2. Clone `aiappsgbb/awesome-gbb`; follow the `azure-tenant-isolation` skill once per workstation.
3. Provision shared infra (items 2–4 above) in the target subscription. Standard `az` commands or a small Bicep are fine; this is one-off and not Zava-specific.
4. (Optional, recommended for customer-facing demos) Run `research-company` then `compose-org` to produce a branded fork of the substrate.
5. Clone `arturcrmbot/zava-control-plane` (or the branded fork from step 4).
6. Follow `skills/zava-workspace-deploy/SKILL.md` step by step:
7. `azd init + azd env new zava`
8. `azd env set ...` for ACR / App Insights / LLM endpoint values (full ~80-var reference in `.env.azd.example` in the repo).
9. `azd up` $\rightarrow$ ~10 min on first run.
10. Smoke-test via the health / workflow / SSE `curl` commands at the bottom of the skill.
11. Turn on Entra Easy Auth on the resulting Container App and lock ingress to your tenant's users.

Explicitly out of scope

Pen-test, threat model, DPIA, Sentinel analytics, immutable WORM audit, DR, SLOs, run-books, replacing embedded KuzuDB with a managed DB, network isolation / private endpoints / WAF, wiring real SaaS systems (Workday, Concur, Greenhouse, ServiceNow, ACS). All of that belongs in a separate productionisation effort once an evaluation passes its

go/no-go — the substrate's own `POC_UNSAFE_FOR_PUBLIC_DEPLOY=1` marker and "Deployment gate" checklist (in the substrate's `README.md`) are the input list for that future scope.